

Loadable Kernel Modules (LKM)

May 5, 2026

Disclaimer

Material and Slides

Some of the material/slides are adapted from various:

- Presentations found on the internet;
- Books;
- Web sites;
- ...

Outline

- 1 Kernel Threads
- 2 Workqueue
- 3 Clock and Time Management
- 4 Timekeeping data structures
- 5 Timers

- 1 Kernel Threads**
- 2 Workqueue
- 3 Clock and Time Management
- 4 Timekeeping data structures
- 5 Timers

What is?

- Kernel thread is a way to implement **background tasks inside the kernel**.
- A background task can be **busy handling asynchronous events or can be asleep, waiting** for an event to occur.
- Kernel threads are similar to user threads, except that **they live in kernel space and have access to kernel functions and data structures**.

Kernel Thread API (I) ¹

- `struct task_struct * kthread_create (int (*threadfn)(void *data), void *data, const char namefmt[], ...);`
 - This function creates and names a kernel thread.
 - But we need to wake up that thread manually.
 - When woken, the thread will run `threadfn()` function with `data` as its argument.
- `int wake_up_process (struct task_struct * p);`
 - This is used to wake up a specific process or thread
- `int kthread_stop (struct task_struct *k);`
 - It stops a thread created by `kthread_create`.
 - Sets `kthread_should_stop` for `k` to return `true`, wakes it and waits for it to exit.

¹/include/linux/kthread.h

Kernel Thread API (II)

- `int kthread_should_stop (void);`
 - If the `kthread_stop` was invoked this function will return true.
- `struct task_struct * kthread_run (int (*threadfn)(void *data), void *data, const char namefmt[], ...);`
 - This is used to create and wake a thread.
 - Convenient wrapper for `kthread_create` followed by `wake_up_process`.

Kernel Thread: Function

- The function thread has as argument a `void *` parameter and should return an `int` value.
- If that thread is a long-run thread, we need to check `kthread_should_stop()` every time, because any function may call `kthread_stop`.
- If any function called `kthread_stop`, that time `kthread_should_stop` will return `true` and the thread function should finish.

```
int kth_function(void *data) {
    int i=0;
    int id = *((int *) data);
    while(!kthread_should_stop()) {
        printk(KERN_INFO "LKM: %s:[%d]:[%d] Kthread function [%d] \n",ENTRY_NAME,
            current->pid, id, i++);
        msleep(id * 1000);
    }
    return 0;
}
```

Kernel Thread Example (I)

```
#include <linux/module.h> /* Needed by all modules */
#include <linux/kernel.h> /* Needed for KERN_INFO */
#include <linux/kthread.h>
#include <linux/delay.h>

#define ENTRY_NAME "LKM_kthread"

int proc_init(void);
void proc_exit(void);
int kth_function(void *pv);
struct task_struct *kth1, *kth2;
int th_id1 = 1, th_id2 = 2;
/*
** Thread Function
*/
int kth_function(void *data){
    ...
}
```

Kernel Thread Example (II)

```
int __init proc_init(void) {  
  
    kth1 = kthread_create(kth_function,&th_id1,ENTRY_NAME);  
    if(kth1) {  
        printk(KERN_INFO "LKM: %s:[%d]:[%d] Kthread Created Successfully\n",ENTRY_NAME,  
            current->pid, th_id1);  
        wake_up_process(kth1);  
        printk(KERN_INFO "LKM: %s:[%d]:[%d] Kthread is executing \n",ENTRY_NAME,  
            current->pid, th_id1);  
    } else {  
        printk(KERN_INFO "LKM: %s:[%d]:[%d] Cannot create kthread\n",ENTRY_NAME,  
            current->pid, th_id1);  
    }  
    kth2 = kthread_run(kth_function,&th_id2,ENTRY_NAME);  
    if(kth2) {  
        printk(KERN_INFO "LKM: %s:[%d]:[%d] Kthread Created Successfully\n",ENTRY_NAME,  
            current->pid, th_id2);  
    } else {  
        printk(KERN_INFO "LKM: %s:[%d]:[%d] Cannot create kthread\n",ENTRY_NAME,  
            current->pid, th_id2);  
    }  
    return 0;  
}
```

Kernel Thread Example (II)

```
void __exit proc_exit(void) {
    if(kth1) {
        kthread_stop(kth1);
        printk(KERN_INFO "LKM: %s:[%d]:[%d] Kthread Stopped Successfully\n", ENTRY_NAME,
               current->pid, th_id1);
    }
    if(kth2) {
        kthread_stop(kth2);
        printk(KERN_INFO "LKM: %s:[%d]:[%d] Kthread Stopped Successfully\n", ENTRY_NAME,
               current->pid, th_id2);
    }
}

module_init(proc_init);
module_exit(proc_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("PBS");
MODULE_DESCRIPTION("The kernel thread example");
```

- 1 Kernel Threads
- 2 Workqueue**
- 3 Clock and Time Management
- 4 Timekeeping data structures
- 5 Timers

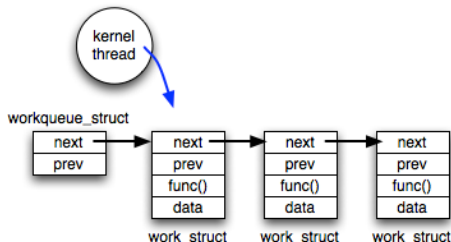
What is?

- Workqueue is a flexible mechanism in the Linux kernel that allow **tasks to be queued and executed by worker threads in the background**.
- It provides a structured way **to defer tasks that do not need to be executed immediately**, enabling better multitasking and resource management.
- Workqueue performs their operations in the context of a process.
 - This means that its execution can be suspended waiting for a specific event from other kernel subsystems.
- The context for workqueue is provided by a specialized kernel thread called a **worker thread**.
 - By default, there are as many worker threads associated with a single work queue as the number of processors on the computer.

How it works?

- The workqueue mechanism can be described **as a system of lists and threads related to them.**
- In the lists that **act as FIFO queues there are structures placed that contain pointers to the functions** performing specific operations.
- Worker threads **remove structures from the list one by one and call the related functions.**
 - Kernel threads responsible for processing tasks from the workqueue.
- This action **ends when the list is emptied.**

Components



- `struct workqueue_struct`: represents a queue for holding tasks.
- `struct work_struct`: defines a specific task to be executed.
- Worker Threads: Kernel threads responsible for processing tasks from the workqueue.
 - A kworker is a kernel thread that performs processing as part of the kernel, especially in the case of interrupts, timers, I/O, etc.

Workqueue API ²(I)

- `struct workqueue_struct *create_workqueue(name);`
 - Work queues are created through a macro called `create_workqueue`, which calls the `alloc_workqueue` function in the background .
- `INIT_WORK(work, func);`
 - The `INIT_WORK` macro provides for the necessary initialization and setting of the handler function
- `int queue_work(struct workqueue_struct *wq, struct work_struct *work);`
 - For enqueueing a work on a work queue

²/include/linux/workqueue.h

Workqueue API (II)

- `int flush_work(struct work_struct *work);` and `int flush_workqueue(struct workqueue_struct *wq);`
 - To flush a particular work item and block until the work is complete, you can make a call to `flush_work`.
 - All work on a given work queue can be completed using a call to `flush_workqueue`.
 - In both cases, the caller blocks until the operation is complete.
- `int cancel_work_sync(struct work_struct *work );`
 - For canceling a work if it is not already executed in a handler.
- `void destroy_workqueue(struct workqueue_struct *) ;`
 - For removing a workqueue

Workqueue API (III)

- Using a kernel-global workqueue.
 - `int schedule_work(struct work_struct *work);`
 - `int schedule_work_on(int cpu, struct work_struct *work);`

Worqueue: Function

- The function thread has as argument a `struct work_struct *work` parameter no return (`void`).
- In order to pass data from the initialization to this function, you must embedded this struct in another data type.

```
typedef struct task {  
    struct work_struct work;  
    int data;  
} task_t;
```

```
void task_function(struct work_struct *work) {  
    task_t *task = container_of(work, task_t, work);  
    int id = task->data;  
    kfree(task);  
}
```

```
tasks[i] = kmalloc(sizeof(task_t), GFP_KERNEL);  
if (tasks[i]) {  
    INIT_WORK(&tasks[i]->work, task_function);  
    tasks[i]->data = i + 1;  
    queue_work(wq, &tasks[i]->work);  
}
```

Workqueue: Example (I)

```
#include <linux/module.h> /* Needed by all modules */
#include <linux/kernel.h> /* Needed for KERN_INFO */
#include <linux/workqueue.h>
#define NTASKS 2

int proc_init(void);
void proc_exit(void);
void task_function(struct work_struct *work);

static struct workqueue_struct *wq;

typedef struct task {
    struct work_struct work;
    int data;
} task_t;

void task_function(struct work_struct *work) {
    task_t *task = container_of(work, task_t, work);
    int id = task->data;
    printk(KERN_INFO "LKM: %s:[%d]:[%d] Work function \n", current->comm, current->pid,
           id);
    kfree(task);
}
```

Workqueue: Example (II)

```
int __init proc_init(void) {
    wq = create_workqueue("LKM_WQ");
    if (wq) {
        task_t *tasks[NTASKS];
        for(int i = 0; i < NTASKS; i++){
            tasks[i] = kmalloc(sizeof(task_t), GFP_KERNEL);
            if (tasks[i]) {
                INIT_WORK(&tasks[i]->work, task_function);
                tasks[i]->data = i + 1;
                queue_work(wq, &tasks[i]->work);
            }
        }
    }
    return 0;
}

void __exit proc_exit(void) {
    flush_workqueue(wq);
    destroy_workqueue(wq);
}

module_init(proc_init);
module_exit(proc_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("PBS");
MODULE_DESCRIPTION("A workqueue kernel module example");
```

- 1 Kernel Threads
- 2 Workqueue
- 3 Clock and Time Management**
- 4 Timekeeping data structures
- 5 Timers

Time Management

- The Linux time management subsystem **manages various time-related activities and keeps track of timing data** such as current time and date, time elapsed since system boot up (system uptime) and timeouts
 - How long to wait for a particular event to be initiated or terminated, locking the system after a timeout period has elapsed, or raising a signal to kill an unresponsive process.
- There are **two types of timing activities** handled by the Linux time management subsystem:
 - **Keeping the current time** and date
 - **Maintaining timers**

Time representation

- Depending on the use cases, time is represented in:
 - **Wall time** (or real time): This is the actual time and date in the real world
 - Such as 07:00 AM, 10 Aug 2017, and is used for timestamps on files and packets sent through the network.
 - **Process time**: This is the time consumed by a process in its life span.
 - It includes the time consumed by the process in user mode and the time consumed by the kernel code when executing on behalf of the process.
 - This is useful for statistical purposes, auditing, and profiling.
 - **Monotonic time**: This is the time elapsed since system bootup.
 - It's ever incrementing and monotonic in nature (system uptime).

Time Measures

- The three times representation (Wall time, Process time, and Monotonic time) are measured in:
 - **Relative time:** This is the time relative to some specific event
 - Such as 7 minutes since system bootup, or 2 minutes since last input from user.
 - **Absolute time:** This is a unique point in time without any reference to a previous event.
 - Such as 10:00 AM, 12 Aug 2017.
 - In Linux, absolute time is represented as the number of elapsed seconds since 00:00:00 midnight of 1 January 1970 (Epoch Time)
- **Wall time is ever incrementing** (unless it has been modified by the user), even between reboots and shutdowns, but **process time and system uptime start from some predefined point in time (usually zero)** every time a new process is created or when the system starts.

Time Management

- There are **a lot of kernel activities that are time-driven**, such as balancing the processor's run-queues, process resource usage accounting, profiling, and so on;
- Time management **is based on some hardware devices that have oscillators of a known fixed frequency, counters, and comparators**;
 - Simpler devices **increment their counters at fixed frequency**, which is in some cases configurable.
 - Other devices **decrement their counters (also at fixed frequency) and are able to issue an interrupt when the counter reaches zero**.
 - More sophisticated devices **are provided with some comparison logic to compare the device counter against a specific value, and are also able to interrupt the processor when those values match**.
- The kernel **keeps track of the flow of time by means of timer interrupts**.

Time related devices (I)

- Linux relies on appropriate hardware devices to maintain time and these hardware devices can be categorized broadly into two types: **system clock** and **timers**.
- **Real-time clock (RTC):**
 - The system clock RTC keeps track of the current time and date.
 - Backed by an additional battery, it continues to tick even when the system is turned off.
 - RTC can raise interrupts periodically (on IRQ8).
 - This feature can be used as an alarm facility, by programming the RTC to raise interrupt when it reaches a specific time.
- **Time Stamp Counter (TSC)**
 - This is a counter implemented in every **x86 microprocessor by means of a 64-bit register called TSC**.
 - It counts the number of clock signals arriving on the **CPU CLK pin**.
 - The current counter value can be read by accessing the TSC register.
 - The number of ticks counted per second can be calculated as $1/(\text{clock frequency})$;
 - For a 1 GHz clock it translates to once every nanosecond.

Time related devices (II)

- **Advanced Configuration and Power Interface (ACPI) Power Management Timer (ACPI PMT)**
 - The ACPI PMT is a **simple counter that has a fixed frequency clock at 3.58 Mhz.**
 - It increments on each tick.
 - The PMT is port mapped;
 - Port-mapped I/O often uses a special class of CPU instructions designed specifically for performing I/O, such as the in and out instructions found on microprocessors based on the x86 architecture.
 - The BIOS takes care of address mapping in the hardware initialization phase during bootup.
 - The PMT is more reliable than the TSC, as it works with a constant clock frequency.
 - The TSC depends on the CPU clock, which can be underclocked or overclocked as per the current load, resulting in time dilation and inaccurate measurements.

Time related devices (III)

- **Programmable Interval Timer (PIT)**

- The PIT keeps on issuing timer interrupts on IRQ0 periodically at approximately 1000 Hz frequency, once every millisecond.
 - This is a **counter that is able to generate an interrupt when it reaches the programmed count**;
 - This periodic interrupt is called **the tick** and the frequency at which it's issued is called the tick rate.
 - The **tick rate frequency is defined by the kernel macro HZ** and is measured in hertz.
- PIT is a global timer, and interrupts raised by it that can be handled by any CPU in an SMP (Symmetric Multi-Processing) system.

Time related devices (IV)

- **Local Advanced Programmable Interrupt Controller (LAPIC)**, which is part of the processor chip.
 - It manages **all interrupts received by the processor either from external devices or internally generated by software**;
 - It is provided with mechanisms for queueing, nesting, and masking interrupts;
 - It is able to send interrupts for external devices as well as to generate interrupts for the respective processor.
 - The APIC timer **works with the bus clock signal**.
 - The APIC timer is quite similar to PIT except that it is local to the CPU, has a 32-bit counter (PIT has a 16-bit one), and works with the bus clock signal (PIT uses its own clock signal).

Time related devices (V)

- **High Precision Event Timer (HPET)**
 - This is a counter and a set of comparators.
 - **Each comparator has a match register and can generate an interrupt** when the associated match register value matches with the main counter value;
 - The HPET works with clock signals in excess of 10 Mhz, issuing interrupts once every 100 nano seconds, hence the name high-precision.
 - HPET implements a 64-bit main counter to count at such a high frequency.
 - Timers can be programmed to generate interrupts either once or periodically.
 - Among all, the HPET is preferable since it allows very short time intervals if present in the system.

Hardware abstraction

- Every system has **at least one clock counter**.
- As with any hardware device in a machine, this counter is also represented and managed by a structure.
- Hardware abstraction is provided by `struct clocksource`
- Different hardware devices provide different clock sources, the capabilities of which vary considerably:
 - TSC is a 64-bit counter that its frequency is equal to CPU.
 - HPET consists of a 64-bit counter of at least 10 MHz frequency.
- All time related devices are managed by Linux kernel as a `struct clocksource` data structure.
 - The main point of the clock source is timekeeping abstraction:
 - In very simple words, **it provides a time value to the kernel**.

struct clocksource (I)

```

struct clocksource {
    u64    (*read) (struct clocksource *cs);
    u64    mask;
    u32    mult;
    u32    shift;
    ...
    const char *name;
    struct list_head list;
    u32    freq_khz;
    int    rating;
    ...
    struct module *owner;
};

```

- `read` it is a function pointer that returns counter value.
- `mask` that allows to ensure that subtraction between counters values from non 64 bit counters do not need special overflow logic.
- `mult` and `shift` are the fields that are base of mathematical functions that convert an abstract device time units of a counter to nanoseconds.
- `name` of a clocksource.

`struct clocksource (II)`

- `rating` field **allows the best registered clock source to be chosen by the clock source management code available for the specified hardware.**
 - The rating may have following value:
 - 1-99 - Only available for bootup and testing purposes;
 - 100-199 - Functional for real use, but not desired.
 - 200-299 - A correct and usable clocksource.
 - 300-399 - A reasonably fast and accurate clocksource.
 - 400-499 - The ideal clocksource.

Getting time

- The `clocksource` data structure shall provide means to translate the provided counter into a nanosecond value as an `unsigned long long` (unsigned 64 bit) number.
- Since this operation may be invoked very often, doing this in a strict mathematical sense is not desirable:
 - The number is taken as close as possible to a nanosecond value using only the arithmetic operations multiply and shift, so in `clocksource_cyc2ns` function you find:

```
static inline s64 clocksource_cyc2ns(u64 cycles, u32 mult, u32 shift)
{
    return ((u64) cycles * mult) >> shift;
}
```

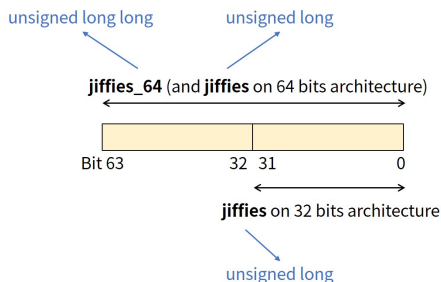
- 1 Kernel Threads
- 2 Workqueue
- 3 Clock and Time Management
- 4 Timekeeping data structures**
- 5 Timers

Jiffies (I)

- The `jiffies` variable **holds the number of ticks elapsed since system bootup.**
- Every time a tick occurs, `jiffies` is incremented by one.
- `jiffies` is a variable of the `unsigned long` type and is therefore only 4 bytes long on 32-bit
 - With a tick rate of 100 HZ, a 32-bit `jiffies` variable would overflow in about 497 days.
 - With HZ increased to 1000, however, that overflow now occurs in just 49.7 days!
- To overcome this issue, a 64-bit variable `jiffies_64` is used instead, which allows for thousands of millions of years before the overflow occurs.

Jiffies (II)

- The kernel uses a trick to prevent efficiency losses when converting between the two different time bases.
 - `jiffies` and `jiffies_64` match in their less significant bits and therefore point to the same memory location or the same register.



- Jiffies can be converted to other time units such as milliseconds, microseconds, and nanoseconds using routines:
 - `jiffies_to_msecs()`, `jiffies_to_usecs()` and `jiffies_to_nsecs()`.

timeval and timespec

- In Linux, the current time is maintained by keeping the **number of seconds elapsed since midnight of January 01, 1970** (called **epoch**);
- The second elements in each of these represent the **time elapsed since the last second in microseconds and nanoseconds**, respectively:

```
struct timespec {
    __kernel_old_time_t tv_sec;  /* seconds */
    long    tv_nsec; /* nanoseconds */
};

struct timeval {
    __kernel_old_time_t tv_sec;  /* seconds */
    __kernel_suseconds_t tv_usec; /* microseconds */
};
```

Timekeeper

- Time (counter value) read from the clock source needs to be accumulated and tracked somewhere;
 - The structure `struct tk_read_base` serves this purpose
- The structure `struct timekeeper` keeps various timekeeping values.
 - It is the primary data structure to maintain and manipulate the timekeeping data for different timelines.
- Timekeeping helper routines `timekeeping_delta_to_ns()` and `timekeeping_get_ns()` help get the correction factor (delta t) between universal time and terrestrial time in nanoseconds.

```
struct tk_read_base {  
    struct clocksource *clock;  
    u64    mask;  
    ...  
};
```

```
struct timekeeper {  
    struct tk_read_base tkr_mono;  
    ...  
};
```

Tick and interrupt handling

- To provide the programming interface, the clock device generating the ticks is abstracted through the structure `struct clock_event_device`

```
struct clock_event_device {  
    void    (*event_handler) (struct clock_event_device *);  
    ...  
};
```

- `event_handler` is the appropriate routine, assigned by the framework to be called by the low-level handler to run the tick.
- Depending on the use cases and the operating mode, `event_handler` could be any of these three routines:
 - `tick_handle_periodic()`, which is the default handler for periodic ticks.
 - `tick_nohz_handler()` is the low-resolution interrupt handler, used in low res mode.
 - `hrtimer_interrupt()` is used in high resolution mode. Interrupts are disabled when it's called.

Tick devices

- The `clock_event_device` abstraction is for the core timing framework;
- The `struct tick_device` abstraction is for tick devices per CPU

```
enum tick_device_mode {
    TICKDEV_MODE_PERIODIC,
    TICKDEV_MODE_ONESHOT,
};

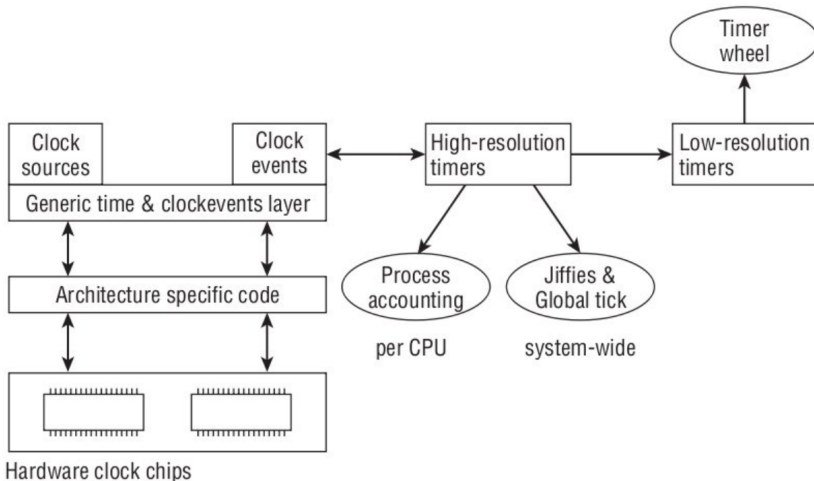
struct tick_device {
    struct clock_event_device *evtdev;
    enum tick_device_mode mode;
};
```

- A `tick_device` could be either periodic or one shot.

- 1 Kernel Threads
- 2 Workqueue
- 3 Clock and Time Management
- 4 Timekeeping data structures
- 5 Timers**

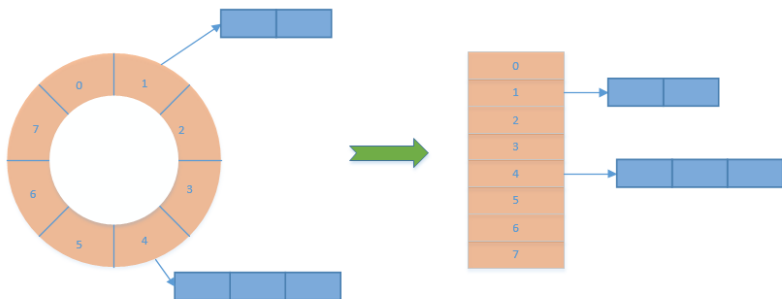
Overview

- A timer **allows a function to be invoked on expiry of a time duration.**

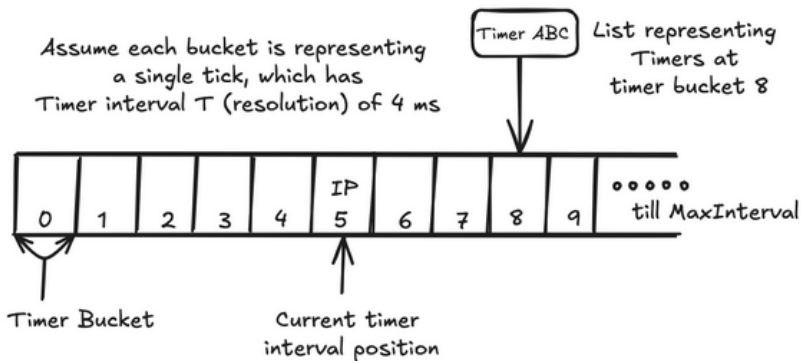


Low-resolution or timer wheel timers (I)

- **Low-resolution timers are centered around a periodic tick which happens at regular intervals.**
 - The low-resolution timer accurately refers to the timer that uses `jiffies` value counting
- Events **can be scheduled to be activated at one of these intervals.**



Low-resolution or timer wheel timers (II)

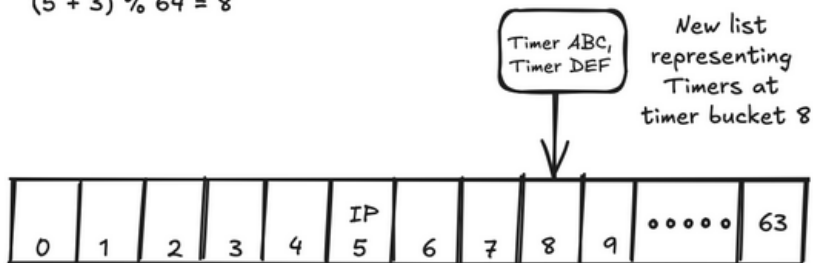


Assume MaxInterval of 256 ms.
Then, $N = 256/4 = 64$

Low-resolution or timer wheel timers (III)

To add a timer 3 units past its current timestamp,
we push it into the list in the timer bucket at the index

$$(5 + 3) \% 64 = 8$$



Low-resolution or timer wheel timers (IV)

- A timer is represented in the kernel as an instance of `struct timer_list`:

```
struct timer_list {  
    struct hlist_node entry;  
    unsigned long expires;  
    void (*function)(struct timer_list *);  
    u32 flags;  
    ...  
};
```

- `expires` is an absolute value in jiffies.
- `entry` is a node for a double linked list, and a callback function.
- The callback function (`*function`) is called to respond to the expiration event of the timer.

Low-resolution or timer wheel timers : API

- `timer_setup (timer, callback, flags)` used to initialize and set up a timer object.
- `int mod_timer (struct timer_list *timer, unsigned long expires)` used to modify a timer's timeout.
 - It is equivalent to and more efficient:

```
del_timer(timer);  
timer->expires = expires;  
add_timer(timer);
```

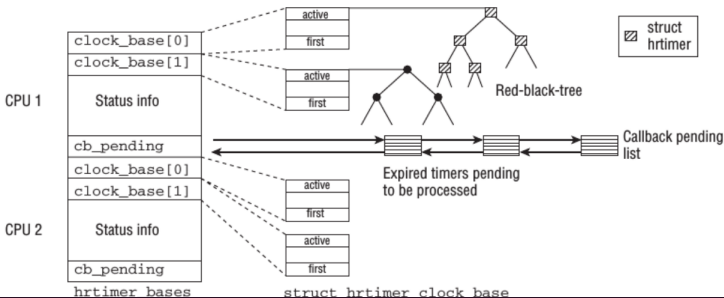
- `int del_timer (struct timer_list * timer)` used to stop a running timer and prevent it's callback function from being executed.
- `msecs_to_jiffies` convert milliseconds to jiffies

High-Resolution Timers (I)

- Two fundamental differences distinguish these timers from low-resolution timers:
 - High-resolution (high-res) timers are time-ordered on a red-black tree.
 - They are independent of periodic ticks.
 - Instead, **they program the hardware timer to fire an interrupt at specific moment.**
 - They do not use a time specification based on `jiffies`, but employ nanosecond time stamps.
- High-resolution timers can be based on two different types of clocks (which are referred to as clock bases).
 - The monotonic clock starts at 0 when the system is booted (`CLOCK_MONOTONIC`).
 - The other clock (`CLOCK_REALTIME`) represents the real time of the system.
 - The latter clock may exhibit skips if, for instance, the system time is changed, but the monotonic clock runs, well, monotonously all the time.

High-Resolution Timers (II)

- For each CPU in the system, a data structure with both clock bases is available.
- Each clock base is equipped with a red-black tree that sorts all pending high-resolution timers.
 - Two clock bases (monotonic and real time) are available per CPU.
 - All timers are sorted by expiration time on a red-black tree.
- Expired timers whose callback handlers still need to be executed are moved from the red-black tree to a linked list.



High-Resolution Timers (III)

- Time is defined as `ktime_t`, which is used to store a time value in nanoseconds.

```
typedef s64 ktime_t;
```

- A generic timer queue is represented by `struct timerqueue_node` and `struct timerqueue_head`:

```
struct timerqueue_node {  
    struct rb_node node;  
    ktime_t expires;  
};  
  
struct timerqueue_head {  
    struct rb_root_cached rb_root;  
};
```

- Manages a simple queue of timers, ordered by expiration time.
- Uses rbtrees for quick list adds and expiration.

High-Resolution Timers (IV)

- A timer is represented by a `struct hrtimer`:

```
struct hrtimer {  
    struct timerqueue_node  node;  
    ktime_t    _softexpires;  
    enum hrtimer_restart    (*function)(struct hrtimer *);  
    struct hrtimer_clock_base *base;  
    ...  
};
```

- `node`: to manage the timer in a rbtree ordered by expiration time.
- `_softexpires`: the absolute earliest expiry time of the `hrtimer`.
The time which was given as expiry time when the timer was armed.
- `function`: timer expiry callback function
- `base`: pointer to the timer base (per cpu and per clock)

High-Resolution Timers: API

- A timer must be initialized with: `void hrtimer_init(struct hrtimer *timer, clockid_t clock, enum hrtimer_mode mode):`
 - `timer` denotes the affected high-resolution timer, `clock` is the clock to bind the timer to (`CLOCK_MONOTONIC` or `CLOCK_REALTIME`), and `mode` specifies if absolute (`HRTIMER_MODE_ABS`) or relative (`HRTIMER_MODE_REL`) time values (relative to the current time) are used.
- Setting a timer: `int hrtimer_start (struct hrtimer * timer, ktime_t time, const enum hrtimer_mode mode):`
 - `timer` denotes the affected high-resolution timer, `time` expiry time and `mode` ...
- `int hrtimer_cancel(struct hrtimer *timer)`
- `int hrtimer_try_to_cancel(struct hrtimer *timer)`
- `int hrtimer_restart(struct hrtimer *timer)`

High-Resolution Timers: Example

```
//Initial phase
//variable declaration
DEFINE_PER_CPU(struct hrtimer, my_hrtimer);
void my_hrtimer_init(){
    int cpu = smp_processor_id();
    struct hrtimer *timer = &per_cpu(my_hrtimer, cpu);
    hrtimer_init(timer, CLOCK_MONOTONIC, HRTIMER_MODE_ABS);
    timer->function = my_hrtimer_callback;
}
//Second phase
void my_hrtimer_start(ktime_t expires){
    int cpu = smp_processor_id();
    struct hrtimer *timer = &per_cpu(my_hrtimer, cpu);
    hrtimer_start(timer, expires, HRTIMER_MODE_ABS);
}
//Third phase
static enum hrtimer_restart my_hrtimer_callback(struct hrtimer *timer){
    ...
    if(cond){
        hrtimer_set_expires(timer, next_expire);
        return HRTIMER_RESTART;
    }
    return HRTIMER_NORESTART;
}
```